

Data Collection.py

This script is a **data collection tool** for a **Sign Language Recognition system**.

👉 It:

- Uses your **webcam**
- Captures **hand gesture images**
- Stores them **class-wise** (A, B, Hello, Thanks, etc.)
- Helps build a **training dataset** for a machine learning / deep learning model

Module Purpose

cv2 OpenCV library → webcam access, image display, saving images

os File & directory handling

time Adding delays between image captures

```
DATA_DIR = './data/imgs'
```

```
NUM_IMAGES_PER_CLASS = 100
```

```
CAPTURE_DELAY = 0.1
```

Meaning:

- **DATA_DIR** → Where images will be saved
- **NUM_IMAGES_PER_CLASS** → Images per gesture/class
- **CAPTURE_DELAY** → Delay between captures (prevents duplicate frames)

👉 This avoids **overfitting** and **identical images**

Function 1: create_directory(path)

```
def create_directory(path):
```

```
    if not os.path.exists(path):
```

```
        os.makedirs(path)
```

Purpose:

- Ensures required folder exists
- Prevents program crash if folder is missing

Why important?

- Dataset is stored **class-wise**
- ML models require **organized datasets**

🔴 Example directory structure:

```
data/  
└─ imgs/  
    └─ A/  
    └─ B/  
    └─ Hello/
```

◆ Function 2: collect_images_for_class(class_name, num_images)

This is the **core function** of the project.

◆ Step 1: Create Class Folder

```
class_dir = os.path.join(DATA_DIR, class_name)
```

```
create_directory(class_dir)
```

- Creates a folder for each gesture
- Example: ./data/imgs/Hello

◆ Step 2: Count Existing Images

```
existing_images = len([f for f in os.listdir(class_dir) if f.endswith('.jpg')])
```

Why?

- Prevents overwriting
- Allows **incremental data collection**

◆ Step 3: Webcam Initialization

```
cap = cv2.VideoCapture(0)
```

- 0 → default webcam

- Returns a **VideoCapture object**

if not cap.isOpened():

- Error handling if camera fails
-

◆ Step 4: Instructions Display

Printed instructions help user:

- Start/stop capture
 - Maintain hand position
 - Avoid motion blur
-

◆ Step 5: Continuous Frame Capture (Main Loop)

while True:

ret, frame = cap.read()

- ret → success flag
 - frame → captured image
-

◆ Step 6: Mirror Effect

frame = cv2.flip(frame, 1)

Why?

- Makes hand movement natural
 - User sees **mirror view**, not reversed
-

◆ Step 7: Overlay Text & UI

cv2.putText(...)

cv2.rectangle(...)

Displays:

- Class name
- Image count
- Capturing status

● Green box → capturing

● Red box → paused

◆ Step 8: Key Controls

`key = cv2.waitKey(1) & 0xFF`

Key Action

s Start / Pause capture

q Quit current class

◆ Step 9: Image Saving Logic

`cv2.imwrite(img_path, frame)`

- Saves frame as .jpg
- File naming: class_index.jpg

📌 Example:

Hello_0.jpg

Hello_1.jpg

◆ Step 10: Delay Between Captures

`time.sleep(CAPTURE_DELAY)`

Prevents:

- Duplicate frames
 - Motion blur
 - Dataset imbalance
-

◆ Step 11: Cleanup

`cap.release()`

`cv2.destroyAllWindows()`

Very important:

- Releases webcam
 - Prevents camera lock
-

◆ Function 3: main()

This controls the **entire flow**.

◆ **Step 1: Intro & Setup**

```
create_directory(DATA_DIR)
```

Ensures base dataset directory exists.

◆ **Step 2: User Inputs Class Names**

```
classes = []
```

User enters:

A

B

Hello

Thanks

Stops when user types done.

◆ **Step 3: Validation Checks**

Prevents:

- Empty names
 - Duplicate classes
 - No classes entered
-

◆ **Step 4: Loop Through Classes**

```
for class_name in classes:
```

```
    collect_images_for_class(class_name, NUM_IMAGES_PER_CLASS)
```

Each class gets:

- Separate folder
 - Equal number of images
-

◆ **Step 5: Final Output**

Shows:

- Total classes

- Dataset location
 - Next step (training)
-

◆ `if __name__ == "__main__":`

```
if __name__ == "__main__":  
    main()
```

Why?

- Ensures `main()` runs **only when file is executed directly**
- Prevents auto execution when imported

VIVA QUESTIONS & ANSWERS (VERY IMPORTANT)

Q1. What is the purpose of this script?

Answer:

This script collects labeled image data for training a sign language recognition model by capturing hand gesture images using a webcam.

Q2. Why do we store images class-wise?

Answer:

Class-wise storage helps supervised learning models associate input images with correct labels during training.

Q3. Why is `cv2.flip(frame, 1)` used?

Answer:

It creates a mirror effect, making hand movement intuitive and user-friendly during data collection.

Q4. Why is a delay (`CAPTURE_DELAY`) required?

Answer:

To avoid capturing identical consecutive frames and to increase dataset diversity.

Q5. What happens if the webcam fails?

Answer:

The program checks `cap.isOpened()` and safely exits if the webcam is unavailable.

Q6. Why do we check existing images?

Answer:

To avoid overwriting data and allow incremental dataset expansion.

Q7. Why is OpenCV preferred here?

Answer:

OpenCV provides real-time video processing, image saving, and GUI support with minimal overhead.

Q8. What kind of ML model will use this data?

Answer:

Typically a CNN (Convolutional Neural Network) such as:

- Custom CNN
 - MobileNet
 - ResNet
 - TensorFlow / PyTorch models
-

Q9. What preprocessing can be added later?

Answer:

- Image resizing
 - Background removal
 - Hand landmark detection
 - Normalization
-

Q10. How can this system be improved?

Answer:

- ROI hand detection
 - Auto capture using motion detection
 - Data augmentation
 - Video-based gesture recognition
-

“This project focuses on building a clean, labeled image dataset for sign language recognition using real-time webcam capture. The script ensures class-wise storage, avoids duplicate data, provides user control, and prepares high-quality input for training deep learning models.”